Course Code: CSA1741 **Course Name: ARTIFICIAL INTELLIGENCE**

Slot: B

ASSIGNMENT 2 – CO2

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: Niranjna devi S I **Reg. No:192519100**

CO2 Coverage: Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimization problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

Problem Context – Autonomous Campus Shuttle

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

- Predict how the three algorithms will behave under different map conditions
- Explain the algorithmic reason for your prediction
- Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Nodes expanded and nodes generated
- Maximum frontier size
- Path cost and whether the path is optimal
- Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

Design Decision

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

Evidence Expected: Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

Problem Context – Exploration Rover in an Unmapped Facility

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

Your Challenge

Design experiments to investigate the online agent under:

- Targets at different distances from the start
- Unreachable targets
- Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

Changed Condition

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

Evidence Expected: Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

Problem Context – Game Engine for a Board Game

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

Your Challenge

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs)
2. One game tree deliberately engineered so that almost no cutoffs occur
3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

- Predict the expected number of nodes visited
- Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs.

Inquiry

Compare prediction with experimental result. Explain:

- Why one tree requires substantially more work than another
- Which characteristics of move ordering create or prevent cutoffs
- What this reveals about evaluating an algorithm using only the branching factor and depth

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

Problem Context – Turn-Based Strategy Game

A game AI must choose its next move by looking ahead in a game with an average of b legal moves per turn.

Your Challenge

Design a depth-limited minimax solution with an evaluation function of your own.

Before implementation, determine:

- How many positions are examined for lookahead depth d
- How the evaluation function is used when the search is cut off
- How the number of terminal evaluations changes with d

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

- Number of nodes expanded and evaluation-function calls
- Execution time, where useful
- Quality of the move chosen (for example, win rate over 20 games against a random player)

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

Evidence Expected: Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

Problem Context – University Exam Timetabling

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

Design Starting Point

You are given only the following principle:

A value may be removed from a variable's domain if some neighboring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

Your Challenge

Convert this principle into a computational procedure. Determine:

- How the constraints are represented
- How a value without support is detected
- Which other pairs must be re-examined after a value is removed
- What condition indicates that the problem has no solution
- How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A timetable instance in which propagation solves or detects failure without any search
- A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

Correctness Inquiry

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

Problem Context – Delivery Tour Planning

A courier must visit n customers exactly once and return to the depot, minimising the total travel distance.

Your Challenge

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

Correctness Check: Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

Growth Investigation

Before experimentation, derive/predict how the number of candidate tours grows with n. Use at least three problem sizes to validate your prediction experimentally.

Local Search Alternative

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

Inquiry

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

Deployment Decision

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest n for which you would deploy exhaustive search, and the point at which you would switch to local search.

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

B. Define the Design Objective – 2 Marks

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

D. Predict Before Implementing – 1 Mark

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

E. Implement and Validate the Redesign – 2 Marks

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

F. Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

MARK DISTRIBUTION – ASSIGNMENT 2

Task	Assessed Area	Marks
Task 1	Uniform-cost, Greedy and A* search	6
Task 2	Online search agent	4
Task 3	Alpha-beta pruning	6
Task 4	Minimax and evaluation functions	6
Task 5	Constraint propagation and CSP	7
Task 6	Exhaustive and local search, optimisation	8
Task 7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Tasks 1–7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

Assessment Verification

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

ASSIGNMENT 2 – CO2: SOLUTIONS

Course: Artificial Intelligence (CSA1741)

TASK 1 – Greedy Best-First, Uniform-Cost and A* Search

Design and assumptions. Represent the campus as a four-neighbour grid. Each legal move costs 1 and blocked cells cannot be entered. The start is S and the destination is G. Uniform-Cost Search (UCS) expands the lowest accumulated cost g(n); Greedy Best-First Search (GBFS) expands the lowest heuristic h(n); A* expands the lowest f(n)=g(n)+h(n).

Admissible heuristic. Use Manhattan distance h(n)=|x_n-x_G|+|y_n-y_G|. In the relaxed problem where all cells are open and movement is restricted to horizontal and vertical steps, this is exactly the shortest path length. Obstacles can only preserve or increase the true path cost, so Manhattan distance never overestimates the actual remaining cost and is admissible. It is also consistent for unit-cost four-neighbour movement.

Contrasting map designs and predictions

- Map A – Open grid with one small obstacle: UCS and A* should find an optimal route. A* should usually expand fewer cells because it prioritises progress toward G. GBFS may be fastest in terms of expansions, but it is not guaranteed to return the shortest path.
- Map B – Maze with a long wall and a misleading corridor: GBFS may follow cells that look close to G but lead to a dead end. UCS systematically expands by path cost and remains optimal. A* combines distance-so-far and estimated remaining distance, so it should usually avoid much of the irrelevant maze while retaining optimality.
- Map C – Edge case with a narrow bottleneck and dead-end branches: GBFS can spend time in a branch whose endpoint is geometrically near G. A* should be guided by g+h, although any heuristic can be less informative when obstacles dominate the geometry. UCS is reliable but may explore many cells at low path-cost layers.

Algorithm outline

UCS: place S in a priority queue with priority g=0. Repeatedly remove the lowest-g node; if it is G, return its path. For every legal neighbour, update it when a lower path cost is found. GBFS uses the same graph-search structure but prioritises h(n) only. A* prioritises g(n)+h(n). For UCS and A*, maintain best-known g values and ignore stale queue entries.

Illustrative comparison

Map	Algorithm	Nodes expanded	Optimal?	Expected behaviour
A: open	UCS	Many	Yes	Explores cost layers
A: open	GBFS	Few	Not guaranteed	Heads toward goal

A: open	A*	Few–moderate	Yes	Uses g+h
B: maze	UCS	High	Yes	Broad systematic exploration
B: maze	GBFS	Variable/high	Not guaranteed	Can enter dead ends
B: maze	A*	Moderate	Yes	Balances cost and heuristic
C: bottleneck	UCS	High	Yes	Unaffected by misleading h
C: bottleneck	GBFS	Variable	Not guaranteed	Sensitive to geometry
C: bottleneck	A*	Moderate/high	Yes	Heuristic helps, but obstacles matter

Interpretation: these are qualitative predictions, not measured node counts. For a fair experiment, use identical maps, start/goal positions, neighbour order and tie-breaking rules. Record nodes expanded, nodes generated, maximum frontier size, final path cost, whether the cost is optimal and elapsed time.

Design decision

Because approximately 95% of trips occur on open roads with only occasional detours, A* with Manhattan distance is the recommended default. It preserves optimality with this admissible heuristic and usually explores fewer nodes than UCS. GBFS may be useful when a quick approximate route is acceptable, but its route may be suboptimal. UCS is appropriate when no useful heuristic is available or as a correctness baseline.

TASK 2 – Online Search Agent

An online agent is suitable when the map is initially unknown, sensing is local, and the rover must act while discovering the environment. Online DFS stores visited cells and the action needed to backtrack. At a cell, it chooses an unexplored legal neighbour; when none remains, it returns to the previous cell. LRTA* can improve repeated behaviour by updating estimated costs from experience.

Experiment design

- Vary target distance: near, medium and far targets in otherwise comparable layouts.
- Vary reachability: one reachable dock and one dock blocked by walls or absent from the connected component.
- Vary environment size and dead-end density: small/large grids with few/many branches.

- For each run, record moves, unique cells visited, revisited cells, success/failure and competitive ratio = online travel cost / optimal cost with complete map knowledge. If the target is unreachable, report failure and do not assign a finite competitive ratio.

Illustrative outcome and interpretation

In an open corridor, online DFS may reach a nearby target with little overhead. In a maze, it may explore many branches and revisit cells while backtracking. An unreachable target can force exploration of the entire reachable component before failure is established. Thus, online search is reasonable for small or moderate unknown environments, emergency exploration, or situations where no map exists and action must begin immediately. It is less attractive in very large environments, dense dead-end mazes, or when strict travel-time limits apply.

Changed condition: with 20 rovers in 20 facilities whose maps remain unchanged for months, an initial mapping phase followed by stored-map planning is preferable. The organisation can share mapping data where appropriate and use an offline planner for later missions. LRTA* may improve repeated trials, but a fixed map makes it wasteful to repeatedly rediscover the same layout. Keep online sensing as a safety mechanism for unexpected obstacles.

Evidence note: run the experiment on generated grids and report measured counts. A useful table has columns: grid size, dead-end density, target status, optimal cost, online moves, revisits, competitive ratio and outcome.

TASK 3 – Engineering Alpha-Beta Pruning

Minimax evaluates a game tree assuming MAX chooses the largest value and MIN chooses the smallest. Alpha-beta pruning returns the same minimax value while skipping branches that cannot affect the final decision. Alpha is the best value found so far for MAX; beta is the best value found so far for MIN. Prune when alpha >= beta.

Three test trees

- Test A – Best moves first: order MAX's strongest child first and MIN's weakest child first. Early bounds become tight, so many later branches are cut off.
- Test B – Worst moves first: order promising moves last. Bounds improve slowly, so few branches are pruned and work approaches ordinary minimax.
- Test C – Equal-valued leaves or random ordering: equal values can still cause cutoffs once alpha reaches beta, while random ordering gives variable performance. Repeat random trials and report the mean and range.

Pseudocode

MINIMAX(node, depth): if terminal or depth=0, return evaluation(node). If MAX node, return the maximum of MINIMAX(child, depth-1) for all children. If MIN node, return the minimum of those values.

ALPHA_BETA(node, depth, alpha, beta): if terminal or depth=0, return evaluation(node). At MAX, initialise v=-infinity; for each child, v=max(v, ALPHA_BETA(child, depth-1, alpha,

beta)); alpha=max(alpha,v); if alpha>=beta, count a cutoff and stop examining children. At MIN, initialise v=+infinity; for each child, v=min(v, ALPHA_BETA(child, depth-1, alpha, beta)); beta=min(beta,v); if beta<=alpha, count a cutoff and stop. Return v.

Analysis

With branching factor b and depth d, minimax visits O(b^d) nodes. Alpha-beta has the same worst-case order when move ordering is poor, but ideal ordering can reduce the effective work toward O(b^(d/2)). The exact node count depends on the tree values and child order, not just b and d. Record visited nodes and cutoffs for both algorithms on the same tree. The minimax values and selected move should agree.

Illustrative prediction: best-first ordering should have the fewest alpha-beta visits and most cutoffs; worst-first ordering should have the most visits and fewest cutoffs; random/equal leaves should fall between or vary by tree. Use actual counter output rather than claiming unrun counts.

TASK 4 – Minimax and the Cost of Examining Every Possibility

Let b be the average number of legal moves per position and d the search depth. A full b-ary tree contains $1+b+b^2+...+b^d = (b^{(d+1)}-1)/(b-1)$ nodes when b is not 1. The number of leaf positions at depth d is b^d. Therefore, increasing depth by one multiplies the leaf count by approximately b.

At a depth cutoff, the evaluation function estimates the desirability of the position from MAX's perspective. Positive values favour MAX, negative values favour MIN, and values near zero indicate a balanced position. The evaluation function is not a proof of the eventual result; its quality affects move quality.

Experimental plan

Test at least three conditions, for example (b=3,d=3), (b=4,d=4), and (b= branching factor of a representative board,d=5). Record nodes expanded, evaluation calls, time, selected move and win rate over 20 games against a random player. Keep the evaluation function and test protocol fixed.

Branching factor b	Depth d	Leaf evaluations b^d	Total nodes (full tree)
3	3	27	40
4	4	256	341
5	4	625	781

These figures are mathematical predictions for a full tree, not observed runtime results. In an implementation, terminal states can stop earlier and legal move counts may vary.

Scalability decision

Define an acceptable response time before testing, for example 1 second per move on the chosen computer. Plain minimax becomes impractical when measured response time exceeds this limit or node counts grow beyond the available computation budget. The main limitation is repeated exploration of many future positions. A redesign should reduce the effective branching factor or avoid repeated work, for example through alpha-beta pruning, better move ordering, iterative deepening or transposition-table reuse.

TASK 5 – Constraint Propagation for University Exam Timetabling

Model each exam as a variable X_i. Its domain D_i contains available time slots. If two exams share students, add a binary constraint X_i != X_j. A value t in D_i has support in neighbouring domain D_j when at least one value u in D_j satisfies the constraint between i and j.

Propagation procedure (arc consistency)

Create a queue containing every directed neighbouring pair (X_i, X_j).

1. Create a queue containing every directed neighbouring pair (X_i, X_j).

2. For each pair, examine every value t in D_i. If no value u in D_j satisfies the constraint, remove t from D_i.

3. If D_i becomes empty, report failure because the current partial assignment cannot be extended.

4. If D_i changed, add arcs (X_k, X_i) for neighbours X_k of X_i other than X_j, because their support may have changed.

5. When the queue is empty, propagation is complete. If variables remain unassigned, choose a variable, assign a candidate value, propagate again, and backtrack if failure occurs.

Pseudocode: AC-3(queue): while the queue is not empty, remove (Xi,Xj); if REVISE(Xi,Xj) removed a value, if D(Xi) is empty return FAILURE; for each neighbour Xk of Xi except Xj, add (Xk,Xi). Return SUCCESS. REVISE removes each x in D(Xi) for which no y in D(Xj) satisfies the constraint.

Test cases and correctness

Test 1: three exams A, B and C all share students pairwise, with domains D(A)=D(B)=D(C)={1,2}. Arc consistency alone does not necessarily empty any domain, so backtracking is still required to prove failure; this is a useful reminder that propagation is not a complete solver for every CSP.

A propagation-only failure example is A and B sharing students, D(A)={1}, D(B)={1}: value 1 in A has no support in B and is removed, causing D(A) to become empty. A solvable

example is A and B sharing students with D(A)={1}, D(B)={1,2}; propagation removes 1 from B and leaves B={2}.

Correctness argument: suppose a value x in D(X_i) is removed because every value in a neighbouring domain D(X_j) conflicts with x. Any complete solution assigning X_i=x would have to assign X_j a value compatible with x, but no such value exists. Therefore x cannot belong to any valid solution and removing it is safe. Domain wipe-out proves that the current search branch has no solution.

Compare backtracking alone against backtracking plus propagation on identical CSP instances. Record search nodes, backtracks, constraint checks and domain wipe-outs. Propagation costs extra checks at each node but can reduce the number of branches explored. Visualise domain sizes after each propagation step with a simple table or bar chart.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth

Represent a tour as an ordering of all customers, starting and ending at the depot. For each candidate ordering, add the distance from the depot to the first customer, between consecutive customers, and from the final customer back to the depot. Keep the lowest-cost tour seen. For a symmetric travelling-salesperson problem, fixing the first customer and treating a tour and its reverse as equivalent reduces duplicate enumeration; there are (n-1)!/2 distinct tours for n customers when n>2. Without symmetry reduction, there are n! customer orders.

Correctness check

For a small four-customer instance, enumerate every distinct tour, calculate its total distance independently, and verify that the retained minimum is no greater than any other enumerated tour. A hand-checkable sample distance matrix should be included in the submitted report, along with the winning route and its computed cost.

Growth investigation

Customers n	Orders n! (no symmetry reduction)	Distinct symmetric tours (n-1)!/2
5	120	12
7	5040	360
9	362880	20160
11	39916800	1814400

The factorial growth means that adding only a few customers can multiply the number of candidates dramatically. Exhaustive search is valuable for small instances because it proves optimality, but it soon exceeds a fixed response-time budget.

Local-search alternative

Use 2-opt local search: begin with a random tour, select two edges, reconnect the route by reversing the segment between them, and accept the change if it reduces tour length. Continue until no improving 2-opt move remains. Repeat from at least 20 random initial tours and retain the best result. For each run, report cost, runtime and percentage gap = 100 × (local cost − optimal cost)/optimal cost.

Local search can stop at a local minimum where no allowed neighbouring move improves the tour, even though a better global tour exists. Plateaus can contain many equal-cost moves. Random restarts or simulated annealing can help, but do not guarantee optimality.

Deployment decision

Choose a response-time budget, such as 1 second, and measure exhaustive enumeration on the target computer. Deploy exhaustive search only for sizes that consistently finish inside the budget; switch to local search when the factorial candidate count makes that limit likely to be exceeded. The exact cut-off n must be selected from measured timings, not assumed from a universal value.

TASK 7 – Diagnose, Redesign and Validate

Selected limitation: exhaustive search for delivery-tour planning. The bottleneck is evaluating a factorial number of customer orders. Most work is spent generating and scoring tours, and the search repeats similar route segments across many permutations.

A. Diagnose the bottleneck

The number of candidate tours grows approximately as (n−1)!/2 for a symmetric tour with the depot fixed and reverse tours treated as duplicates. This causes a rapid increase in distance calculations and memory/time demand. The original method has little reuse of partial-route calculations.

B. Design objective

Objective: reduce the number of complete tours evaluated while preserving the exact optimal answer on small and medium test cases. Measure complete tours evaluated, partial branches pruned, distance calculations, runtime and final route cost.

C. Revised strategy

Use branch-and-bound. Build routes incrementally and maintain the cost accumulated so far. Calculate a valid lower bound on the cheapest possible completion of each partial route. If accumulated cost plus the lower bound is at least the best complete tour found, stop exploring

that branch because it cannot improve the incumbent. Seed the incumbent with a nearest-neighbour tour to make the bound useful early.

Pseudocode: set best_cost=infinity; obtain an initial complete tour and set it as the incumbent. SEARCH(partial_route, unvisited, cost): if unvisited is empty, add return-to-depot cost and update incumbent if better; otherwise compute a valid lower bound for completing this partial route; if cost+bound >= best_cost, count a prune and return; for each candidate next customer, recurse with that customer appended and its edge cost added. Return the best complete route.

A valid lower bound is essential for correctness. For example, a conservative bound can add the cheapest required outgoing connection for each remaining customer and a connection back toward the depot, provided the bound is constructed so it never exceeds the true remaining cost. Branch-and-bound trades additional bound calculations and bookkeeping for fewer complete tours. Its worst case remains exponential/factorial, especially when bounds are weak.

D. Prediction

Prediction: branch-and-bound will evaluate fewer complete tours than exhaustive enumeration on instances where the incumbent and lower bound are informative, while returning the same optimal cost. It may offer little improvement on difficult instances with weak bounds.

E. Implementation and validation

Run both methods on the same distance matrices. Verify that both return the same optimal tour cost for small instances whose exhaustive optimum is known. Record complete tours evaluated, branches pruned, distance calculations and runtime. Also test a case where the bound is weak to demonstrate that pruning is instance-dependent.

F. Final defence

The redesign is successful if it preserves the optimum and reduces measured search work on the selected instances. Report the reduction as 100 × (exhaustive evaluations − redesigned evaluations)/exhaustive evaluations. Costs include lower-bound computation and extra code complexity. Exhaustive search remains preferable for tiny instances because it is simple and provides a transparent baseline; branch-and-bound is more useful as problem size increases.